



timm documentation

Quickstart ▾



Join the Hugging Face community

and get access to the augmented documentation experience

[Sign Up](#)

to get started



Quickstart

[Copy page](#)

This quickstart is intended for developers who are ready to dive into the code and see an example of how to integrate `timm` into their model training workflow.

First, you'll need to install `timm`. For more information on installation, see [Installation](#).

```
pip install timm
```

Load a Pretrained Model

Pretrained models can be loaded using [create_model\(\)](#).

Here, we load the pretrained `mobilenetv3_large_100` model.

```
>>> import timm

>>> m = timm.create_model('mobilenetv3_large_100', pretrained=True)
>>> m.eval()
```

Note: The returned PyTorch model is set to train mode

[Ask HuggingChat](#)

plan to use it for inference.

List Models with Pretrained Weights

To list models packaged with `timm`, you can use `list_models()`. If you specify `pretrained=True`, this function will only return model names that have associated pretrained weights available.

```
>>> import timm
>>> from pprint import pprint
>>> model_names = timm.list_models(pretrained=True)
>>> pprint(model_names)
[
    'adv_inception_v3',
    'cspdarknet53',
    'cspresnext50',
    'densenet121',
    'densenet161',
    'densenet169',
    'densenet201',
    'densenetblur121d',
    'dla34',
    'dla46_c',
]
```

You can also list models with a specific pattern in their name.

```
>>> import timm
>>> from pprint import pprint
>>> model_names = timm.list_models('*resnext*')
>>> pprint(model_names)
[
    'cspresnet50',
    'cspresnet50d',
    'cspresnet50w',
    'cspresnext50',
    ...
]
```



Fine-Tune a Pretrained Model

You can finetune any of the pre-trained models just by changing the classifier (the last layer).

```
>>> model = timm.create_model('mobilenetv3_large_100', pretrained=True, num_classes=NUM_F
```

To fine-tune on your own dataset, you have to write a PyTorch training loop or adapt `timm`'s [training script](#) to use your dataset.

Use a Pretrained Model for Feature Extraction

Without modifying the network, one can call `model.forward_features(input)` on any model instead of the usual `model(input)`. This will bypass the head classifier and global pooling for networks.

For a more in depth guide to using `timm` for feature extraction, see [Feature Extraction](#).

```
>>> import timm
>>> import torch
>>> x = torch.randn(1, 3, 224, 224)
>>> model = timm.create_model('mobilenetv3_large_100', pretrained=True)
>>> features = model.forward_features(x)
>>> print(features.shape)
torch.Size([1, 960, 7, 7])
```

Image Augmentation

To transform images into valid inputs for a model, you can use `timm.data.create_transform()`, providing the desired `input_size` that the model expects.

This will return a generic transform that uses reasonable defaults.

```
>>> timm.data.create_transform((3, 224, 224))
Compose(
  Resize(size=256, interpolation=bilinear, max
  CenterCrop(size=(224, 224))
```



```
ToTensor()  
Normalize(mean=tensor([0.4850, 0.4560, 0.4060]), std=tensor([0.2290, 0.2240, 0.2250]))  
)
```

Pretrained models have specific transforms that were applied to images fed into them while training. If you use the wrong transform on your image, the model won't understand what it's seeing!

To figure out which transformations were used for a given pretrained model, we can start by taking a look at its `pretrained_cfg`

```
>>> model.pretrained_cfg  
{'url': 'https://github.com/rwightman/pytorch-image-models/releases/download/v0.1-weights  
'num_classes': 1000,  
'input_size': (3, 224, 224),  
'pool_size': (7, 7),  
'crop_pct': 0.875,  
'interpolation': 'bicubic',  
'mean': (0.485, 0.456, 0.406),  
'std': (0.229, 0.224, 0.225),  
'first_conv': 'conv_stem',  
'classifier': 'classifier',  
'architecture': 'mobilenetv3_large_100'}
```

We can then resolve only the data related configuration by using `timm.data.resolve_data_config()`.

```
>>> timm.data.resolve_data_config(model.pretrained_cfg)  
{'input_size': (3, 224, 224),  
'interpolation': 'bicubic',  
'mean': (0.485, 0.456, 0.406),  
'std': (0.229, 0.224, 0.225),  
'crop_pct': 0.875}
```

We can pass this data config to `timm.data.create_transform()` to initialize the model's associated transform.



```
>>> data_cfg = timm.data.resolve_data_config(model.pretrained_cfg)
>>> transform = timm.data.create_transform(**data_cfg)
>>> transform
Compose(
  Resize(size=256, interpolation=bicubic, max_size=None, antialias=None)
  CenterCrop(size=(224, 224))
  ToTensor()
  Normalize(mean=tensor([0.4850, 0.4560, 0.4060]), std=tensor([0.2290, 0.2240, 0.2250]))
)
```

Note: Here, the pretrained model's config happens to be the same as the generic config we made earlier. This is not always the case. So, it's safer to use the data config to create the transform as we did here instead of using the generic transform.

Using Pretrained Models for Inference

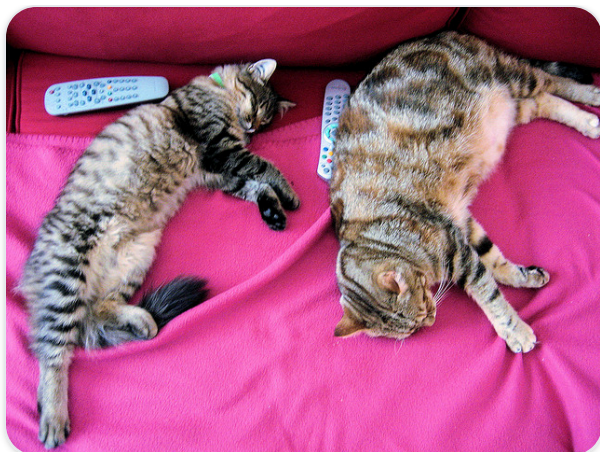
Here, we will put together the above sections and use a pretrained model for inference.

First we'll need an image to do inference on. Here we load a picture of a leaf from the web:

```
>>> import requests
>>> from PIL import Image
>>> from io import BytesIO
>>> url = 'https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/'
>>> image = Image.open(requests.get(url, stream=True).raw)
>>> image
```

Here's the image we loaded:





Now, we'll create our model and transforms again. This time, we make sure to set our model in evaluation mode.

```
>>> model = timm.create_model('mobilenetv3_large_100', pretrained=True).eval()
>>> transform = timm.data.create_transform(
    **timm.data.resolve_data_config(model.pretrained_cfg)
)
```

We can prepare this image for the model by passing it to the transform.

```
>>> image_tensor = transform(image)
>>> image_tensor.shape
torch.Size([3, 224, 224])
```

Now we can pass that image to the model to get the predictions. We use `unsqueeze(0)` in this case, as the model is expecting a batch dimension.

```
>>> output = model(image_tensor.unsqueeze(0))
>>> output.shape
torch.Size([1, 1000])
```

To get the predicted probabilities, we apply softmax to the output. This leaves us with a tensor of shape `(num_classes,)`.

```
>>> probabilities = torch.nn.functional.softmax(
>>> probabilities.shape
```



```
torch.Size([1000])
```

Now we'll find the top 5 predicted class indexes and values using `torch.topk`.

```
>>> values, indices = torch.topk(probabilities, 5)
>>> indices
tensor([281, 282, 285, 673, 670])
```

If we check the imagenet labels for the top index, we can see what the model predicted...

```
>>> IMAGENET_1k_URL = 'https://storage.googleapis.com/bit_models/ilsvrc2012_wordnet_lemma'
>>> IMAGENET_1k_LABELS = requests.get(IMAGENET_1k_URL).text.strip().split('\n')
>>> [{'label': IMAGENET_1k_LABELS[idx], 'value': val.item()} for val, idx in zip(values,
[{'label': 'tabby, tabby_cat', 'value': 0.5101025700569153},
 {'label': 'tiger_cat', 'value': 0.22490699589252472},
 {'label': 'Egyptian_cat', 'value': 0.1835290789604187},
 {'label': 'mouse, computer_mouse', 'value': 0.006752475164830685},
 {'label': 'motor_scooter, scooter', 'value': 0.004942195490002632}]]
```

[Update on GitHub](#)

[← Home](#)

[Installation →](#)

